

Informe de Laboratorio #2

Comunicación y Sincronización entre Procesos

FIFO – Shared Memory

Sistemas Operativos

Universidad Santiago De Cali

Juan Pablo Terán

Juan Sebastián Giraldo

Juan David Victoria

Comunicación y sincronización entre procesos usando FIFO y Memoria Compartida

1. Decisiones de Diseño

Se desarrollaron dos versiones del sistema de pedidos entre procesos:

- **Opción A: FIFO (First In, First Out)**
- **Opción B: Memoria Compartida (Shared Memory, SHM)**

Ambas versiones aplican mecanismos de sincronización para evitar condiciones de carrera y garantizar la integridad de los datos.

1.1 FIFO

- Se implementaron dos archivos FIFO: uno para **pedidos** y otro para **respuestas**.
- Cada cliente escribe su pedido en `fifo_pedidos` y espera confirmaciones en `fifo_respuestas`.
- El proceso **cocina** lee los pedidos en orden de llegada y responde a cada cliente según su `cliente_id` y estado.
- Se evitaron interferencias entre clientes usando filtrado por ID y estado del mensaje.

1.2 Memoria Compartida

- Se usó una **estructura circular** para manejar la cola de pedidos.
- Se implementó un arreglo de pedidos compartidos con índices `head` (para la cocina) y `tail` (para los clientes).
- Se utilizaron **tres semáforos**:
 - **Semáforo 0**: Exclusión mutua (mutex).
 - **Semáforo 1**: Control de espacios disponibles.
 - **Semáforo 2**: Control de pedidos disponibles.

2. Respuestas a Preguntas Propuestas

2.1 ¿Cómo evitar que dos procesos cliente escriban al mismo tiempo en la misma posición?

- **En SHM:** Se utiliza el semáforo mutex. Cada cliente ejecuta `sem_wait(0)` antes de entrar en la sección crítica y `sem_signal(0)` al salir.
- **En FIFO:** El sistema operativo asegura escritura secuencial, por lo que no se requiere sincronización adicional.

2.2 ¿Qué pasa si la cocina lee un pedido mientras el cliente aún lo escribe?

- **En SHM:** El cliente debe hacer `sem_signal(2)` después de escribir. La cocina espera con `sem_wait(2)`.
- **En FIFO:** El sistema gestiona lecturas/escrituras completas. Sin embargo, una mala gestión de bloques podría causar lectura incompleta.

2.3 ¿Cómo sincronizar para que los clientes esperen su confirmación?

- **En FIFO:** Cada cliente filtra las respuestas en `fifo_respuestas` por su `cliente_id` y estado.
- **En SHM:** Se puede usar un estado dentro del pedido y polling, o un semáforo por cliente para mayor precisión (aunque más complejo).

2.4 ¿Qué pasa si la cocina es más lenta que los clientes?

- **En SHM:** El semáforo de espacios (`sem_wait(1)`) bloquea a los clientes cuando la cola está llena.
- **En FIFO:** El buffer puede llenarse y causar bloqueos si la cocina no lee a tiempo.

2.5 Diferencias prácticas entre FIFO y SHM

Característica	FIFO	Memoria Compartida
Facilidad	Más simple de implementar	Requiere manejo de semáforos
Eficiencia	eficiente	Más rápido en concurrencia
Sincronización	Implícita en el sistema operativo	Control total con semáforos
Flexibilidad	Limitada	Mayor personalización
Escalabilidad	Baja	Alta

2.6 ¿Cómo escalar a múltiples cocinas sin duplicar pedidos?

- **En SHM:** Los procesos cocina deben coordinar con `sem_wait(2)` y `sem_wait(0)`. Solo uno debe incrementar el índice head.
- **En FIFO:** Se recomienda usar un FIFO por cocina o un despachador que distribuya los pedidos para evitar duplicación.

3. Análisis de Rendimiento y Robustez

FIFO:

- Fácil de implementar.
- Depende del sistema operativo para evitar condiciones de carrera.
- Riesgo de bloqueo si el buffer se llena.

Memoria Compartida:

- Excelente rendimiento en sistemas concurrentes.
- Requiere una implementación cuidadosa.
- Ofrece control total sobre el flujo de datos.

4. Conclusión

Ambos mecanismos cumplen con la comunicación entre procesos, pero tienen enfoques distintos:

- **FIFO:** Ideal para aplicaciones simples con bajo volumen de datos.

- **SHM:** Recomendado para sistemas concurrentes que requieren eficiencia, escalabilidad y control detallado.

La elección depende de los objetivos y limitaciones del sistema a implementar.